CS100 Introduction to Programming
Spring 2025
Midterm Exam

Instructors: Lan Xu, Yuexin Ma

Time: 21 May 2025, 8:15–10:15

INSTRUCTIONS

Please read and follow the following instructions:

1. The examination paper comprises **four major questions**. The level of difficulty **does not necessarily increase** in sequential order. Please **manage your time wisely**.
2. **This exam is relatively lengthy. For each question, concise answers are expected. If you encounter unfamiliar material, move on promptly rather than spending excessive time. Detailed explanations are not required; grading will be based strictly on key language points.**
3. Each of the four primary questions is preceded by a sign **[C]** or **[C++]** indicating whether the question pertains to the C or C++ programming language, respectively.
4. You have **120 minutes** to answer the questions.
5. You are permitted to bring **at most three (3) standard-sized A4 cheatsheets**, which may be **printed or hand-written**. Cheatsheets that do not meet these requirements will be **confiscated**.
6. You should write the answer to every problem in the dedicated box **clearly**.
7. You should write **your name and your student ID** as indicated on the top of **each page** of the exam sheet.
8. You are **not allowed to bring any electronic devices**, including regular calculators and electronic watches.
9. You are **not allowed to discuss or share anything** with others during the exam.

Name	
Student ID	
School Email	

(This page is intentionally left blank.)

1. (14 points) [C] Input and Output

Imagine you are the instrcutor for a programming exam. Before students can continue, you want each one to honestly promise not to cheat. You write the following C program, which asks the student to type a specific sentence and enter their student number:

```c
#include <stdio.h>

int main(void) {
  int num;
  scanf("I promise not to cheat in this exam. My student number is %d.", &num);
  printf("Student %d finishes his/her promise.\n", num);
  return 0;
}
```

The key part of this program is how `scanf` requires the input to exactly match the format string. If you are not familiar with this behavior, here is an excerpt from *en.cppreference.com*:

> `int scanf(const char *format, ...);`
>
> Reads data from `stdin`, matches it to the format string, and stores the results in the provided variables.
>
> ---
>
> ### Parameters
>
> - **format** — a pointer to a null-terminated string describing the expected input format
> - **...** — the variables that will receive the input values
>
> The format string may include:
>
> - Non-whitespace characters except `%`: **each character in the format string must exactly match the input, or the function fails.**
> - Whitespace characters
> - Conversion specifications (like `%d`, `%s`)
>
> ---
>
> ### Return value
>
> The number of input items successfully assigned (may be zero if no matches occurred), or EOF if an input failure occurs before any assignment.

(1) Answer the following questions to explore how this program behaves. You may consult the "Parameters" section in the excerpt above, which explains the meaning and use of each argument in the function prototypes.

i. (1') What is the exact input you must enter for this program to work correctly? Use `'\n'` to indicate where you should press Enter. *Write your answer exactly as you would type it, but replace the sample student number with your own.*

Solution:

`I promise not to cheat in this exam. My student number is 2024533000.\n`

(Note that the ending `\n` should not be omitted.)

Every character in the line—including the spaces, the final period, and the exact capitalization—must appear exactly as written because `scanf` treats ordinary characters in its format string as fixed text that must match the input stream before it proceeds to the `%d` conversion. If even one character is missing or altered, the matching phase stops, the conversion is never reached, and the variable `num` remains uninitialized.

ii. (1') After you enter the above, what will the program print? Use `'\n'` to represent a newline in your answers.

Solution:

`Student 2024533000 finishes his/her promise.\n`

(Note that the ending `\n` should not be omitted.)

(2) (2') This simple program is not reliable: if the student mistypes even one character, the program may not behave as expected. The improved version below will keep asking until the student enters the sentence correctly. Fill in the blanks to check for errors and ignore bad input. (Hint: check the return value of `scanf`.)

```c
#include <stdio.h>

int main(void) {
  int num, matched, c;
  do {
    matched = scanf(
        "I promise not to cheat in this exam. My student number is %d.", &num);
    if (__matched != 1__) {

      /* Discard the rest of the line */
      while ((c = getchar()) != '\n' && c != EOF);

      printf("Oops—there was a typo. Please try again!\n");
    }
  } while (__matched != 1__);

  printf("Thank you! Student %d has promised honestly.\n", num);
  return 0;
}
```

Solution: Both blanks should be `matched != 1`. Since `scanf` returns the number of successful conversions, testing `matched != 1` catches any mismatch or missing number, triggers the error handling, and repeats the loop until `scanf` returns 1.

(3) Now imagine you are the instructor at the front of the classroom. Before you let any student make their promise, you must show you are really the instructor. After you log in, you will call up each student, one at a time, and keep track of how many students have made their promise.

```c
#include <stdio.h>
#include <string.h>
#include <stdbool.h>

/* Instructor login check */
bool verify_instructor(void);

/* Student promise routine */
void check_promise(void);

int main(void) {
  /* 1. Check that you are the instructor */
  if (!verify_instructor()) {
    printf("Access denied. You shall not pass!\n");
    return 1;
  }

  /* 2. Ask each student to make their promise */
  int count = 0;
  char choice;
  do {
    check_promise();
    count++;
    printf("Next student, please? (y/n): ");
    // the leading space skips leftover newlines
    if (scanf(" %c", &choice) != 1)
      break;
  } while (choice == 'y' || choice == 'Y');

  /* 3. Print the total number of students */
  printf("All done! Total students: %d\n", count);

  return 0;
}
```

The only missing part is the instructor login. Before any student can make a promise, you must type your username and secret passkey. The function must:

1. Read your input safely
2. Remove the trailing newline character

3. Check that the input matches an approved username and passkey

Important: DO NOT attempt to fill in the blanks in the code below until you have carefully read and followed the steps and hints provided in the sub-questions and documentation excerpts.

```
1  bool verify_instructor(void) {
2    char user[50], pass[50];
3
4    printf("Username: ");
5    if (!__fgets(user, sizeof user, stdin)__)
6      return false;
7    user[ strcspn(user, "\n") ] = '\0';
8
9    printf("Passkey: ");
10   if (!__fgets(pass, sizeof pass, stdin)__)
11     return false;
12   pass[ strcspn(pass, "\n") ] = '\0';
13
14   const char *names[] = {"Yuexin Ma", "Lan Xu"};
15   const char *keys[]  = {"8888", "6666"};
16
17   unsigned n = sizeof(names) / sizeof (char *);
18   for (unsigned i = 0; i != n; ++i) {
19     if (__strcmp(user, names[i]) == 0__ && __strcmp(pass, keys[i]) == 0__)
20       return true;
21   }
22   return false;
23 }
```

i. (2') To read user input into a buffer safely, use the function `fgets`. Below is an excerpt from *en.cppreference.com* describing the usage of `fgets`. Consult this documentation and fill in the missing arguments on **lines 5 and 10** above.

> `char *fgets(char *str, int count, FILE *stream);`
>
> Reads up to count - 1 characters from the file stream into the string array str. Stops if it reads a newline character (which is also stored in str), or if it reaches the end of the file. On success, puts a null character after the last character in str.
>
> ---
>
> **Parameters**
>
> - `str` — pointer to a char array
> - `count` — maximum number of characters to write (usually the array length)
> - `stream` — file stream to read from

Return value

- Returns `str` on success, or a null pointer on failure.

Solution: By passing `sizeof user` (resp. `sizeof pass`) as the count argument, `fgets` will read at most `count - 1` characters into the array—leaving exactly one slot for the terminating `'\0'`—and store any newline it encounters. Specifying `stdin` ensures the input comes from the standard input stream.

ii. (2') To ensure the buffers are null-terminated, use `strcspn` as illustrated. Consult the excerpt from *en.cppreference.com* below, which describes `strcspn`, and fill in the blanks on **lines 7 and 12** above accordingly.

```
size_t strcspn(const char *dest, const char *src);
```

Returns the length of the initial part of `dest` that does not have any character from `src`.

Parameters

- `dest` — pointer to the string to check
- `src` — pointer to the string of characters to stop at

Return value

The length of the initial segment that has only characters not found in `src`.

Example

```
#include <string.h>
#include <stdio.h>
int main(void) {
  const char *string = "abcde312$#@";
  const char *invalid = "*$#";
  size_t valid_len = strcspn(string, invalid);
  if(valid_len != strlen(string))
    printf("'%s' contains invalid chars starting at position %zu\n",
           string, valid_len);
}
```

Output:

```
'abcde312$#@' contains invalid chars starting at position 8
```

Solution: The call to `strcspn(buf, "\n")` returns the index of the first newline character in the buffer; if no newline was stored—because the input exactly filled the array—the function instead yields the buffer's current length. Overwriting the byte at that index with `'\0'` simultaneously removes any trailing newline and guarantees that both `user` and `pass` remain properly null-terminated, thereby ensuring every subsequent `strcmp` operates on well-formed C strings.

iii. (2') Finally, compare the username and passkey using `strcmp`. Refer to the excerpt from *en.cppreference.com* below, which explains the function `strcmp`, and fill in the appropriate expression on **line 19** above.

> `int strcmp(const char *lhs, const char *rhs );`
>
> Compares two null-terminated byte strings lexicographically. The sign of the result is the sign of the difference between the values of the first pair of characters (both interpreted as unsigned char) that differ in the strings being compared. The behavior is undefined if lhs or rhs are not pointers to null-terminated byte strings.
>
> ---
>
> **Parameters**
>
> - `lhs, rhs` - pointers to the null-terminated byte strings to compare
>
> ---
>
> **Return value**
>
> - Negative value if lhs appears before rhs in lexicographical order.
> - Zero if lhs and rhs compare equal.
> - Positive value if lhs appears after rhs in lexicographical order.

Solution: You can write the test either as

```
strcmp(user, names[i]) == 0 && strcmp(pass, keys[i]) == 0
```

or more tersely

```
!strcmp(user, names[i]) && !strcmp(pass, keys[i])
```

In either form, you're checking that each `strcmp` returns 0 (i.e. the input string exactly equals the stored string), so the condition is true only when both username and passkey match.

(4) Now, let us combine the student promise logic and the counter into a single function. By declaring the counter variable as `static` within `check_promise()`, its value is preserved across multiple function calls. This approach simplifies the code and makes it easier to manage:

```
1  void check_promise(bool finalize) {
```

```
2     __static__ __int__ count = 0;  // persists across calls
3     if (finalize) {
4       printf("Total students processed: %d\n", count);
5       return;
6     }
7     count++;
8     // other parts
9   }
10  int main(void) {
11    // ...
12    do {
13      check_promise(__false__);
14      // ...
15    } while ( /* user chooses to continue */ );
16    check_promise(__true__);
17    return 0;
18  }
```

i. (1') Fill in the blanks on **line 2** above with the correct keywords to declare the counter variable so that its value is preserved across multiple calls to the function.

ii. (1') When does the variable `count` get its first value? Select only **ONE** correct answer.

A. Before the program starts.

B. After the program starts but before `main()` begins.

C. The first time `check_promise()` runs.

D. Each time the program reaches **line 2**.

> **Solution:** It first gets initialized (to 0) before `main()` begins, as part of static initialization.

iii. (2') The boolean parameter `finalize` is used to determine when the total number of students should be printed. Fill in the blanks on **line 13** and **line 16** above so that, after processing all students, the total count is displayed.

> **Solution:** Passing `false` makes the function do `count++`; passing `true` makes it print the accumulated count without incrementing.

2. (37 points) [C, C++] Pointers and Arrays

The department's lightweight *GradeBook* utility maintains a list of student names and their corresponding quiz grades. These are currently stored in two separate arrays.

(1) Below is an example of an initial roster.

```
char *roster[] = { "Alice", "Bob", "Cara", NULL };
```

The structure of this array is similar to a null-terminated string. Please answer the following questions regarding the C string representation.

i. (1') Choose **ALL** the correct statements about the declaration `char str[] = "string";`:

A. It fails to initialize `str`.

B. It initializes `str` to a fixed-size array with length 6.

C. It initializes `str` to a fixed-size array with length 7.

D. It initializes `str` to a fixed-size array whose size is unknown.

E. It initializes `str` to a dynamically allocated array.

Solution: By the C standard, a string literal is an array object of type `char[N]`, where `N` equals the number of characters plus one for the implicit `'\0'`. Hence, `"string"` has type `char[7]`. When you write `char str[] = "string";`, the compiler allocates `str` as a seven-element array `'s','t','r','i','n','g','\0'`, thereby guaranteeing correct null-termination for all standard C string routines.

ii. (3') Additionally, string literals can be bound in the following ways: `char *str1 = "string1";` or `const char *str2 = "string2";`. Which of these implementations (if any) allows modification of the string? Which implementation (`str1` or `str2`) is better? Please provide a brief explanation.

Solution: Neither declaration permits safely modifying the characters of a string literal since literals reside in read-only memory; any write through either `str1` or `str2` invokes undefined behavior. Declaring the pointer as `const char *str2 = "string2";` is therefore strongly preferred: the `const` qualifier both documents the immutability of the literal and enables the compiler to catch any accidental attempts to alter its contents at compile time.

(2) To determine the number of students in the roster when the size of the array is unknown, we can use the `sizeof` operator to calculate the byte size of the array. Alternatively, we can add a `NULL` pointer at the end of the array (e.g., `roster[3] == NULL`) and traverse it. Please answer the following questions about the function `count_names`.

```
size_t count_names(char *const a[]) {
  size_t n = 0;
  while (a[n] != NULL)
    n++;
  return n;
}
```

i. (2') The type `size_t` is used to store the result of the `sizeof` operator. Is `size_t` a typedef (alias) for an existing type, or is it a separate, distinct type? Is it signed or unsigned?

Solution: `size_t` is a typedef (alias) for an unsigned integer type, implementation-defined, used to represent sizes and counts.

ii. (1') If `count_names` is called with the `roster` array defined as above, what value will the function return?

Solution: 3.
`count_names` increments until it finds the sentinel `NULL`; there are three non-NULL entries ("Alice", "Bob", "Cara") before the terminator.

iii. (6') Please specify the types of the following expressions. Write the type exactly as it appears in C, instead of vague descriptions such as "a pointer" or "a C language statement."

α) `n++`

β) `roster`

γ) `roster[0]`

δ) `&(roster[0])`

ϵ) `a`

ζ) `a[0]`

α) `size_t`

β) `char *[4]`

γ) `char *`

δ) `char **`

ϵ) `char *const *`

ζ) `char *const`

Solution:

α) `n++`
Post-increment produces a value of the `same built-in type` (here `size_t`).

β) `roster`
It is **initialized with exactly four elements**, so its static type is `char *[4]` – an **array of 4 "pointer-to-char"s**. (Read outside-in: first notice the array `[4]`, then the `*` "pointer to")

γ) `roster[0]`
Indexing removes the array layer, leaving `char *`: a **pointer to char**.

δ) `&(roster[0])`
Prefix `&` adds one level, giving `char **`: a **pointer to (pointer to char)**. (Right→left twice: first `*` → pointer, second `*` → pointer to that)

ϵ) `a`
In `char *const a[]`, the array decays to a pointer, yielding `char *const *` – a **pointer to a const pointer to char**. (Right→left: outer `*` "pointer to"; inner `*const` "const pointer"; finally `char`)

ζ) `a[0]`
Selecting the first element keeps the inner qualifier: `char *const`, a **const pointer to char**. (Right→left: `*const` "const pointer" → `char`)

(3) Since the original `roster` array is designed to remain immutable, a writable copy is required. Below is an auxiliary function that performs a deep copy of the roster on the heap, along with an excerpt from *en.cppreference.com* explaining the C string library function `strcpy` for reference.

```c
#include <stdlib.h>

char **clone_roster(char *const src[]) {
  size_t n = count_names(src);
  char **dst = malloc(   (n + 1) * sizeof(char * const)   );
  for (size_t i = 0; i < n; i++) {
    dst[i] = malloc(   strlen(src[i]) + 1   );
    strcpy(dst[i], src[i]);
  }
  dst[n] = NULL;
  return dst;
}
```

`char *strcpy( char *dest, const char *src );`

Copies the null-terminated byte string pointed to by `src`, including the null terminator, to the character array whose first element is pointed to by `dest`. Undefined behavior occurs if the `dest` array is not large enough, or if the strings overlap. Additionally, the behavior is undefined if either `dest` is not a pointer to a character array or `src` is not a pointer to a null-terminated byte string.

Parameters

- `dest` – pointer to the character array to write to
- `src` – pointer to the null-terminated byte string to copy from

Return value

Returns a pointer to `dest`.

i. In C, copying an array of pointers like `char *roster[]` can be done as a *shallow copy* or a *deep copy*. A **shallow copy** copies only the pointers, so both arrays point to the same strings. A **deep copy** creates new memory for each string and copies the contents, making the arrays fully independent. For each statement below about copying `roster`, indicate True (T) or False (F):

α) (1') A shallow copy of the `roster` array duplicates only the pointer values, not the strings themselves.

α) True

Solution: A shallow copy simply copies each pointer in the array, so both the original and the copy reference the same underlying string objects.

β) (1') If both the original and the shallow-copied `roster` are later freed, this will always be safe and correct.

β) False

Solution: Since both arrays point to the same strings, freeing both will result in undefined behavior of double-free.

γ) (1') A deep copy of the `roster` array allocates new storage for each string and duplicates their contents, which ensures that modifying or freeing one array has no impact on the other.

γ) True

Solution: Deep copying allocates fresh memory for each string and copies the contents, so the new array owns independent memory.

ii. (4') Fill in the blanks in the above code snippet for `clone_roster`. Below is the excerpt of `strlen` from *en.cppreference.com*, which may help when filling in the blanks:

```
size_t strlen(const char *str);
```

Returns the length of the given null-terminated byte string, i.e., the number of characters in a character array whose first element is pointed to by `str`, up to but not including the first null character. Undefined behavior occurs if `str` is not a pointer to a null-terminated byte string.

Parameters

- `str` – pointer to the null-terminated byte string to be examined

Return value

The length of the null-terminated byte string `str`.

Solution:

- The expression `(n + 1) * sizeof(char * const)` reserves enough bytes for `n` pointers plus the terminating `NULL` pointer, ensuring the cloned roster array has space for every entry and its sentinel.
- The expression `strlen(src[i]) + 1` computes the exact length of the source string and adds one for the null terminator, guaranteeing that `strcpy` can safely copy all characters including the final `'\0'`.

(4) Since the copied `roster` uses dynamically allocated memory, you must also write a function to free all the memory when it is no longer needed.

```c
#include <stdlib.h>
/* Frees everything returned by clone_roster(): */
void free_roster(char **roster) {
  size_t n = count_names(src);
  for (size_t i = 0; i != n; ++i)
    free(roster[i]);
  free(roster);
}
```

i. (2') Please fill in the blanks to complete the function implementation.

Solution: Each inner `malloc` must be paired with a corresponding `free`; after all strings are released the outer array itself is deallocated.

ii. (1') What will be the program's state if we mistakenly call `free_roster` on the original `roster`? Select only **ONE** correct answer.

A. Undefined behavior.
B. Unspecified behavior.
C. Compile-time error.
D. Runtime error.
E. The program runs normally.

Solution: Calling `free` on a pointer that was not returned by `malloc`, `calloc`, or `realloc` results in undefined behavior.

(5) Recall that the roster functions as a mini *Gradebook*, which requires the ability to store student grades. To achieve this, we introduce an additional array, `score`, to record the grades for each student across all subjects:

```c
int score[3][3] = { { 7, 9, 8 }, /* Alice */
                    {10, 6, 7 }, /* Bob */
                    {6, 8, 10 }}; /* Cara */
```

i. (1') What is the value of `score[1][2]`?

Solution: `7`.
Row index `1` corresponds to Bob, column `2` (0-based) is his third quiz: the literal initializer `{10, 6, 7}` puts `7` in that position.

ii. (2') Select **ALL** expressions whose values are equivalent to the address of Bob's first quiz grade.

A. `&score[1][0]`
B. `score + 1`
C. `*(score + 1)`
D. `score[1]`

Solution: ABCD.

- `&score[1][0]`: Directly yields an `int*` to the element at row 1, column 0.
- `score + 1`: Advances to the second row; though its type is `int (*)[3]`, the numeric address equals `&score[1][0]`.
- `*(score + 1)`: Dereferences the row pointer to an array, which decays to `int*` pointing at element [1][0].
- `score[1]`: Defined as `*(score + 1)`, and likewise decays to the same `int*`.

iii. (2') Write the pointer-arithmetic expression, using only `p`, `row` and `col`, that yields the same value as `score[row][col]`, given `int score[M][N];` and `auto p = &score;`

Solution: `*(*(p + row) + col)`

Note: The original prompt suggested using `int **p = (int **)score;`, which cannot reproduce the indexing semantics of a contiguous `int[M][N]` array. Since no valid `int **` declaration can achieve this, the question itself is flawed and ***all students will receive full credit***.

(6) [C++] The current approach for managing the student roster and their quiz scores is not yet complete. In real-world scenarios, the number of students cannot be predicted in advance. As a result, both `roster` and `score` should be dynamically allocated on the heap to accommodate varying sizes, which introduces memory management challenges. In contrast, C++ provides a simpler solution for such challenges through the use of STL containers. Below is the C++ equivalent representation for `roster` and `score`:

```
struct Student {
  std::string name;
  std::vector<int> scores;
};
```

i. (1') Initialize an object of `struct Student` named `stu_1` with the information for the student "Cara".

Solution:

```
Student stu_1{"Cara", {6, 8, 10}};
```

ii. (1') Initialize a `std::vector` called `roster` using the given `stu_1` object as its only element.

Solution:

```
std::vector<Student> roster{stu_1};
```

iii. (2') STL containers follow the RAII (Resource Allocation Is Initialization) paradigm, which simplifies manual memory management by acquiring all resources in constructors and releasing them in destructors. Please explain how this approach allows the binding of an object's lifetime to its scope and helps avoid memory-related issues, particularly memory leaks.

Solution: STL containers (e.g., `std::vector`, `std::string`) implement RAII by confining resource acquisition to the constructor and resource release to the destructor. Because these special member functions are invoked automatically at scope entry and exit—including during exception unwinding—the resource lifetime is precisely aligned with the object's lexical lifetime. This automatic, deterministic cleanup eliminates explicit `new`/`delete` pairs, precludes memory leaks, and guarantees strong exception safety.

(7) (5') [**C++**] Many situations require the `score` array to work in coordination with the `roster`. For example, the following C function `praise_students` is designed to print the names of all students whose quiz scores are all greater than or equal to 7.

```
#include <stdio.h>

void praise_students(char *const roster[], int **scores, size_t cols) {
  size_t rows = count_names(roster);
  for(size_t i = 0; i != rows; ++i){
    int all_seven_or_above = 1;
    for(size_t j = 0; j != cols; ++j){
      if(scores[i][j] < 7){
        all_seven_or_above = 0;
        break;
      }
    }
    if(all_seven_or_above)
      printf("Well done, %s! All your grades are 7 or above.\n", roster[i]);
  }
}
```

Please rewrite the `praise_students` function in C++ based on the `roster` data structure. Be sure to maintain const correctness and avoid unnecessary copies of the data.

```cpp
#include <iostream>

void praise_students( const std::vector<Student> & roster) {
  for ( const auto &student : roster) {
    bool all_seven_or_above = true;
    for ( int score : student.scores ) {
      if ( score < 7 ) {
        all_seven_or_above = false;
        break;
      }
    }

    if (all_seven_or_above) {
      std::cout << "Well done, " << student.name
                << "! All your grades are 7 or above.\n";
    }
  }
```

Solution:

- Parallel C arrays (`roster` & `scores`) are merged into an RAII aggregate: `struct Student {std::string name; std::vector<int> scores; }` held in a `const std::vector<Student> &` to avoid copying and forbid mutation.
- Index-based `for` loops become range-for (`const auto & student` and `int score`) for clarity and safety.
- `printf` is replaced by type-safe `std::cout`; headers switch from `<stdio.h>` to `<iostream>`.

3. (20 points) [C++] Constructors and the Builder Pattern

You have previously encountered code structures similar to the following:

```
class Student {
private:
  std::string name_;
public:
  /* TODO: Implement the constructor here. */

  const std::string &name() const { return name_; }
  std::string &name() { return name_; }
};
```

Notice that an appropriate constructor for `Student` has yet to be implemented. There are two typical ways to implement such a constructor:

- **Option 1**

  ```
  explicit Student(const std::string &name) : name_{name} {}
  ```

- **Option 2**

  ```
  explicit Student(std::string name) : name_{std::move(name)} {}
  ```

(1) (2') Briefly explain the advantage of marking this constructor as `explicit`.

Solution: If the constructor is not marked `explicit`, implicit conversions from `std::string` to `Student` would be permitted, which is undesirable and may lead to unintended behaviors.

(2) (5') Analyze how each constructor implementation behaves when passed arguments of different value categories (such as lvalues and rvalues). Based on your analysis, determine which implementation is better.

Solution:

- **Option 1** (`explicit Student(const std::string& name) : name_{name}{}`)
  - *Rvalue argument.* The temporary string binds directly to the reference parameter `name`; `name_` is then copy-constructed from that temporary, so exactly *one* copy occurs.
  - *Lvalue argument.* The parameter binds directly to the original object without an initial copy, after which `name_` is copy-constructed from `name`, resulting in one well-justified copy.
- **Option 2** (`explicit Student(std::string name) : name_{std::move(name)}{}`)
  - *Rvalue argument.* The temporary is move-constructed into the parameter, and the member is then move-constructed from the parameter. No copies are made.

– *Lvalue argument.* The argument is first copied into the parameter; afterward, `name_` is move-constructed from `name`. This incurs one copy followed by one inexpensive move.

Owing to its "pass-by-value, then move" pattern, **Option 2** minimizes redundant work—eliminating copies for rvalues and keeping lvalue calls to a single copy—making it the preferred choice in modern C++17 code.

(3) Once a suitable constructor is provided, you can instantiate `Student` objects as follows:

```
// Instance 1.
Student student0 = "Shaw";
// Instance 2.
Student student1 = student0;
// Instance 3.
Student student2 = std::move(student0);
```

i. In C++, expressions are classified as *lvalue*, *xvalue*, or *prvalue*. For the right-hand side of each assignment above (Instances 1–3), specify the value category by filling in the blank below.

α) (1') (Instance 1) `"Shaw"` **lvalue**

Solution: A string literal is stored in a fixed location and has static storage duration, so the expression denotes an object identity (i.e. a glvalue), making it an lvalue.

β) (1') (Instance 2) `student0` **lvalue**

Solution: Any named variable designates an object with identity; thus the expression `student0` is an lvalue.

γ) (1') (Instance 3) `std::move(student0)` **xvalue**

Solution: `std::move` casts its argument to an rvalue reference, yielding an "expiring" glvalue that still has identity but may be moved from, i.e. an xvalue.

ii. For Instance 2 and Instance 3 above, write down the declarations of the constructors that are invoked.

α) (1') Instance 2 `Student(const Student &);`

Solution: Copy constructor of `Student`.

β) (1') Instance 3 `Student(Student &&);`

Solution: Move constructor of `Student`.

(4) (2') Consider the scenario where the `Student` class is extended to include additional member variables such as ID number and email address:

```
class Student {
private:
  std::string name_;
  int id_;
  std::string email_;

public:
  Student(const std::string &name, int id, const std::string &email)
    : name_(name), id_(id), email_(email) {}

  const std::string &name() const { return name_; }
  std::string &name() { return name_; }
  const int &id() const { return id_; }
  int &id() { return id_; }
  const std::string &email() const { return email_; }
  std::string &email() { return email_; }
};
```

While a constructor with three parameters is still manageable, constructors can quickly become complicated and hard to use when a class has many data members. For example, imagine a `Student` class that stores not only a name, ID, and email, but also fields such as GPA, phone number, address, and exam scores:

```
class Student {
  std::string name_;
  int id_;
  std::string email_;
  double gpa_;
  std::string phone_;
  std::string address_;
  std::vector<int> exam_scores_;

public:
  Student(const std::string &name, int id, const std::string &email,
          double gpa, const std::string &dob, const std::string &phone,
          const std::string &address, const std::vector<int> &exam_scores)
    : name_(name), id_(id), email_(email), gpa_(gpa), phone_(phone),
      address_(address), exam_scores_(exam_scores) {}
};
```

To address this challenge, C++ developers commonly utilize the **Builder Pattern**, which mitigates the drawbacks of constructors with an excessive number of parameters. The objective is to support an interface that enables the following construction style:

```
Student student = Student{}.name("Shaw")
                           .id(2024533000)
                           .email("shaw1@shanghaitech.edu.cn");
```

In this example, a default-constructed `Student` object is created, and member functions `name(...)`, `id(...)`, and `email(...)` are called in sequence to assign values to the respective fields. The following code fragments are member functions of the `Student` class. Complete the code below so that this form of chained initialization is supported:

```
class Student {
  std::string name_;
  int id_{};
  std::string email_;
public:
  Student() = default;

  const std::string &name() const { return name_; }
  std::string &name() { return name_; }
  const int &id() const { return id_; }
  int &id() { return id_; }
  const std::string &email() const { return email_; }
  std::string &email() { return email_; }

  Student &name(const std::string &value) {
    name_ = value;
    return *this;
  }

  Student &id(int value) {
    id_ = value;
    return *this;
  }

  Student &email(const std::string &value) {
    email_ = value;
    return *this;
  }
};
```

Solution:

1. `id_ = value;` This statement assigns the integer argument `value` to the private data member `id_` of the current `Student` object, thereby updating its ID field.
2. `return *this;` Here, `*this` denotes the object on which the member function was called. Returning it by reference (`Student&`) allows calls to this setter to be chained in a fluent style, e.g., `student.id(1234).email("foo@bar");`.

(5) The current implementation lacks robustness: it does not guarantee that all required fields are initialized. For instance, a user might forget to set the `email` field, resulting in an incomplete or invalid `Student` object. To address this, we seek a mechanism that ensures all necessary fields are set before a `Student` instance can be created.

To achieve this, we introduce a separate `StudentBuilder` type, which manages the construction process and performs runtime validation to ensure every required field is assigned. The goal is to support the following usage:

```
Student student = StudentBuilder{}.name("Shaw")
                                  .id(2024533000)
                                  .email("shaw1@shanghaitech.edu.cn");
```

The `StudentBuilder` class is used to help create a `Student` object step by step. It allows users to set each required field individually, using a chain of method calls. Before a `Student` object can be created, `StudentBuilder` checks that all necessary information has been provided. This design ensures that no incomplete or invalid `Student` objects can be constructed, helping to prevent errors caused by missing data.

Important: DO NOT attempt to fill in the blanks in the code below until you have carefully read and followed the steps and hints provided in the sub-questions and documentation excerpts.

```
1  class Student {
2    __friend class StudentBuilder__;  // Your declaration of StudentBuilder here
3    std::string name_;
4    int id_{};
5    std::string email_;
6
7  public:
8    Student() = default;
9
10   const std::string &name() const { return name_; }
11   std::string &name() { return name_; }
12   const int &id() const { return id_; }
13   int &id() { return id_; }
14   const std::string &email() const { return email_; }
15   std::string &email() { return email_; }
```

```
16 };
```

```
1  class StudentBuilder {
2    Student student_;
3  public:
4    StudentBuilder & name(const std::string &value) {
5      student_.name_ = value ;
6      return *this ;
7    }
8
9    // The implementations of id and email are similar and omitted for brevity.
10   ... id(int value) { ... }
11   ... email(const std::string &value) { ... }
12
13   operator Student() {
14     assert( student_.id_ != 0 && !student_.name_.empty()
15             && !student_.email_.empty() );
16
17     return std::move( student_ );
18   }
19 };
```

i. (1') To allow `StudentBuilder` to access the private members of `Student`, declare `StudentBuilder` as a friend class of `Student`. Complete the declaration in the blank at `Student`, **Line 2**.

Solution: The line

```
friend class StudentBuilder;
```

declares `StudentBuilder` as a `friend` of `Student`. As a result, all member functions of `StudentBuilder` are allowed to access `Student`'s private and protected members (such as `name_`, `id_`, and `email_`).

ii. (2') The `StudentBuilder` class supports method chaining for setting fields, similar to the member functions in `Student`. Complete the implementation of the `name` method in `StudentBuilder` by filling in the blanks in `StudentBuilder`, from **Line 4** to **Line 7**.

Solution: By returning a reference to the same `StudentBuilder`, each setter call yields the builder itself, enabling fluent chaining:

```
StudentBuilder{}
  .name("Shaw")   // returns the builder…
  .id(2024533000) // …so we can immediately call id(…)
  .email("…");    // …and then email(…)
```

Without returning '*this', the chain would break after the first call.

iii. (3') To enable conversion from a `StudentBuilder` object to a `Student` object, implement the

type conversion operator. **In particular, ensure that the following invariants hold before construction:**

- The `name` field must be a non-empty string.
- The `id` field must not be zero.
- The `email` field must be a non-empty string.

Fill in the blanks for this operator in `StudentBuilder`, from **Line 14** to **Line 18**.

Solution: By returning `std::move(student_)`, the operator invokes 'Student''s move constructor, transferring ownership of its internal resources (the strings and integer) in constant time rather than copying them.

4. (29 points) [C++] Inheritance and Polymorphism

Expressions constitute the fundamental components of computation in programming languages by combining operators and operands to produce values. In C++, each expression is categorized by its value category (lvalue vs. rvalue) and by its **arity**, which specifies the number of operands an operator accepts. The most common arities are:

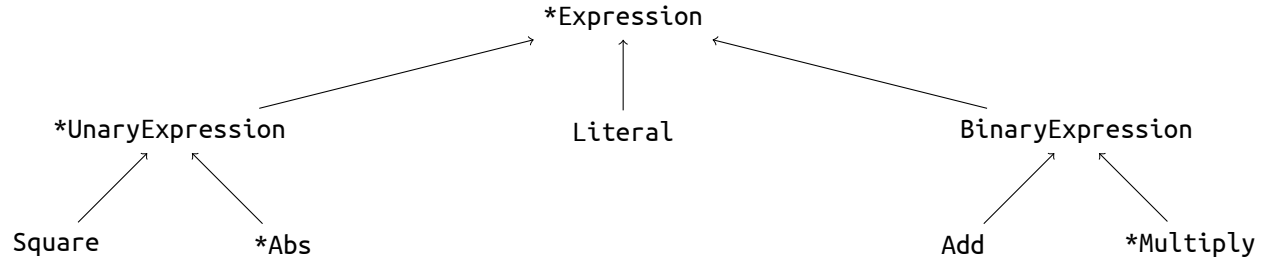
- **Literals** (arity 0): constants with fixed values (e.g. `42`).
- **Unary expressions** (arity 1): an operator applied to one operand (e.g. `-x`).
- **Binary expressions** (arity 2): an operator applied to two operands (e.g. `x + y`).

Although source code may seem to combine more than two operands—such as `a + b + c`—the compiler actually interprets these expressions as a series of nested binary operations:

$$\texttt{a} + \texttt{b} + \texttt{c} \;\equiv\; (\texttt{a} + \texttt{b}) + \texttt{c},$$

This nesting reflects the recursive nature of expressions: both unary and binary expressions take subexpressions as operands, which themselves can be literals, unary expressions, or binary expressions. Consequently, complex expressions are constructed by layering these components from literals at the leaves up to the root expression.

To model this naturally recursive structure, we define **an abstract base class `Expression`**, from which the following concrete subclasses inherit:



Please note the following details represented in the diagram:

- Each node corresponds to a C++ class in the expression hierarchy.
- Arrows indicate public inheritance, pointing from the derived class to its base class.
- Nodes marked with an asterisk (`*`) indicate classes whose definitions are incomplete; you may be asked to supply their implementations in the following questions.
- The class named `Abs` represents the mathematical "absolute value" operation, which returns the non-negative value of a number (e.g., $\text{Abs}(-3) = 3$ and $\text{Abs}(5) = 5$).

The following code blocks present the class definitions in the expression hierarchy. Please **read each definition carefully** before proceeding to the exercises below.

```
class Square : public UnaryExpression {
public:
```

```
  explicit Square(Expression *sub_exp) : UnaryExpression(sub_exp) {}

  ~Square() override {}

  int evaluate() const override {
    return exp->evaluate() * exp->evaluate();
  }
  std::string to_string() const override {
    return "Square(" + exp->to_string() + ")";
  }
  std::string getType() const override {
    return "Square";
  }
};
```

```
class BinaryExpression : public Expression {
protected:
  Expression *exp1;
  Expression *exp2;

public:
  BinaryExpression(Expression *sub_exp1, Expression *sub_exp2)
    : exp1(sub_exp1), exp2(sub_exp2) {}

  ~BinaryExpression() override {}
};
```

```
class Add : public BinaryExpression {
public:
  Add(Expression *sub_exp1, Expression *sub_exp2)
    : BinaryExpression(sub_exp1, sub_exp2) {}

  ~Add() override {}

  int evaluate() const override {
    return exp1->evaluate() + exp2->evaluate();
  }
  std::string to_string() const override {
    return "(" + exp1->to_string() + " + " + exp2->to_string() + ")";
  }
  std::string getType() const override {
    return "Add";
  }
};
```

```
class Literal : public Expression {
```

```
private:
  int num;

public:
  explicit Literal(int n) : num(n) {}

  ~Literal() override {}

  int evaluate() const override {
    return num;
  }
  std::string to_string() const override {
    return std::to_string(num);
  }
  std::string getType() const override {
    return "Literal";
  }
};
```

(1) Review the definition of class `Add` and indicate whether each statement below is True (T) or False (F):

i. (1') The `public` specifier on the second line is redundant, as members of a `class` default to public access.

i. False

Solution: In a `class`, every member that lacks an explicit access label is *private* by default; only a `struct` defaults to *public*. Therefore an explicit `public:` is required to make the inherited members of `Add` accessible outside the class.

ii. (1') Applying the `override` keyword to `Add::evaluate()`, `Add::to_string()`, and `Add::getType()` is optional but strongly recommended, as it clarifies that these methods override base-class virtual functions.

ii. True

Solution: The language does not *require* the `override` specifier because it would have broken pre-C++11 code, yet adding it instructs the compiler to verify that a function truly overrides a virtual base-class signature, catching name- or signature-mismatches at compile time.

iii. (1') In the body of `Add::evaluate()`, calling `exp1->evaluate()` (or `exp2->evaluate()`) will always cause infinite recursion.

iii. False

Solution: Each pointer `exp1`/`exp2` refers to a *different* dynamic object that may be a `Literal`, another `Add`, etc. The virtual call dispatches to the concrete type of that object; it does not re-enter `Add::evaluate()` on the `this` object, so no infinite recursion

occurs. This is a direct consequence of dynamic (run-time) polymorphism in C++.

iv. (1') Both `Add::evaluate()` and `Add::to_string()` can access the protected members `exp1` and `exp2` declared in the base class `BinaryExpression`.

iv. True

Solution: A derived class may read or modify a protected data member that it inherits from its base, whether that member belongs to the current object or to any other instance of the same derived type.

v. (1') Since `~Add()` has an empty body and performs no work, it could be declared `const`.

v. False

Solution: The C++ grammar forbids cv-qualifiers (`const`, `volatile`) on a destructor; every destructor implicitly acts on a modifiable object, even if its body is empty. Attempting to add `const` is ill-formed and rejected by the compiler.

(2) The following is a partial definition of class `Multiply`. As a sibling class of `Add` (both inherit from `BinaryExpression`), you may consult `Add`'s definition for reference while completing the exercises.

```
class Multiply : public BinaryExpression {
public:
  Multiply(Expression *sub_exp1, Expression *sub_exp2)
    : BinaryExpression(sub_exp1, sub_exp2) {}

  ~Multiply() override {}

  int evaluate() const override {
    return ___exp1->evaluate() * exp2->evaluate()___;
  }
  std::string to_string() const override {
    return ___"(" + exp1->to_string() + " * " + exp2->to_string() + ")"___;
  }

  std::string getType() const override {
    return "Multiply";
  }
};
```

i. (2') Fill in the blanks above to complete the implementations of `Multiply::evaluate()` and `Multiply::to_string()`.

Solution: Please refer to the sibling class `Add`, another direct subclass of `BinaryExpression`.

ii. Determine whether each statement below is Correct or Incorrect. If you judge a statement as Incorrect, briefly explain why.

α) (2') Since `BinaryExpression` has its own constructor, you can create a `BinaryExpression` object directly.

> **Solution: Incorrect.**
> `BinaryExpression` still inherits three pure-virtual functions (`evaluate`, `to_string`, `getType`) from `Expression`, so it is an *abstract* class and cannot be instantiated.

β) (2') Because `Add` and `Multiply` do not add any new data members, their constructors are inherited from the base class.

> **Solution: Incorrect.**
> Ordinary constructors are *not* inherited in C++; a derived class must declare its own or explicitly use `using Base::Base`.

γ) (2') The statement `Add add_object;` is valid and creates an instance of `Add`.

> **Solution: Incorrect.**
> `Add` already has a user-defined constructor, so the compiler does **not** supply a default one; therefore the declaration `Add add_object;` is ill-formed. Under the *Rule of Three/Five*, whenever a class provides any special member (constructor, destructor, copy/move op.), the remaining ones should be declared or explicitly `deleted` as well.

(3) Next, consider the class `UnaryExpression` and its subclasses.

i. (3') Provide the full definition of `UnaryExpression`, including its constructor, destructor, and any necessary member functions.
Hint: `UnaryExpression` is the counterpart of `BinaryExpression` in this hierarchy. Whereas `BinaryExpression` manages two operand expressions named `exp1` and `exp2`, `UnaryExpression` contains a single operand pointer named `exp`, consistent with the design shown in `Square`.

> **Solution:**
>
> ```
> class UnaryExpression : public Expression {
> protected:
>   Expression *exp;
>
> public:
>   explicit UnaryExpression(Expression *sub_exp) : exp(sub_exp) {}
>   ~UnaryExpression() override {}
> };
> ```
>
> The single-operand pointer `exp` mirrors `exp1`/`exp2` in `BinaryExpression`. No virtual functions are overridden here, so the class remains abstract.

ii. (3') Similarly, provide the full definition of class `Abs`, implementing all necessary member functions as required, but do not introduce any additional data members.
Hint: Refer to `Square` as an example of a fully defined subclass of `UnaryExpression`.

Solution:

```
class Abs : public UnaryExpression {
public:
  explicit Abs(Expression *sub_exp) : UnaryExpression(sub_exp) {}
  ~Abs() override {}

  int evaluate() const override {
    int v = exp->evaluate();
    return v < 0 ? -v : v;
  }
  std::string to_string() const override {
    return "Abs(" + exp->to_string() + ")";
  }
  std::string getType() const override {
    return "Abs";
  }
};
```

(4) (4') Complete the definition of the abstract base class `Expression`. Considering the interface requirements imposed by `BinaryExpression`, `UnaryExpression`, and `Literal`, declare the appropriate member functions as pure virtual.

Solution:

```
class Expression {
public:
  virtual ~Expression() {}  // virtual dtor for safe polymorphic delete
  virtual int evaluate() const = 0;
  virtual std::string to_string() const = 0;
  virtual std::string getType() const = 0;
};
```

A virtual destructor guarantees that deleting an object through a base-class pointer runs the derived destructor first. Declaring the three member functions as *pure virtual* (`= 0`) turns `Expression` into an *abstract class*; every concrete subclass must override them.

(5) Based on your completed hierarchy, answer the following:

i. (1') Choose **ALL** the abstract classes (those containing at least one pure virtual function).
A. `Expression` B. `BinaryExpression` C. `UnaryExpression` D. `Literal` E. `Add`
F. `Multiply` G. `Square` H. `Abs`

Solution: All three marked classes declare or inherit a pure-virtual, so they cannot be instantiated.

ii. (1') Choose **ALL** the polymorphic classes (those declaring at least one virtual function).

A. `Expression` B. `BinaryExpression` C. `UnaryExpression` D. `Literal` E. `Add` F. `Multiply` G. `Square` H. `Abs`

Solution: Any class that possesses a virtual member (directly or by inheritance) is polymorphic and supports dynamic dispatch.

(6) (4') Consider the following program, which uses `parseExpression()` to create an `Expression` object from its input and then outputs the results of `to_string()` and `evaluate()`:

```
#include <iostream>

Expression *parseExpression(const std::string &);

int main(int argc, char *argv[]) {
  if (argc != 2)
    std::cout << "Illegal input." << std::endl;
  else {
    Expression *exp = parseExpression(argv[1]);
    if (exp != nullptr) {
      std::cout << exp->to_string() << " = "
                << exp->evaluate() << std::endl;
      delete exp;
    } else
      std::cout << "Parsing failed." << std::endl;
  }
  return 0;
}
```

Here, `parseExpression` is a function (implementation omitted) that takes a string representing an arithmetic expression and recursively constructs the corresponding expression tree using classes such as `Literal`, `Add`, `Multiply`, `Abs`, and `Square`. The function operates by:

1. Splitting the input by `'+'` or `'*'` operators not enclosed in parentheses, recursively parsing the parts into binary expressions.
2. Detecting unary functions like `Abs(...)` or `Square(...)` and recursively parsing their inner expressions.
3. Removing surrounding parentheses and parsing the enclosed expression.
4. Returning a `Literal` node if the string represents a number.

The examples below serve to clarify the operation of this complex function:

- Input `"3"` returns a `Literal` with value 3.
- Input `"Abs(-3)"` returns an `Abs` expression whose operand is a `Literal` of `-3`.
- Input `"2+3"` returns an `Add` expression with two `Literal` children 2 and 3.

Suppose the classes above have been compiled into an executable program named `Expression`. For each of the following function calls, write exactly what is printed to standard output. Be sure to include the full output line as it would appear.

i. `./Expression Square(5*5)`

Solution: `Square((5*5)) = 625`

ii. `./Expression 5*Abs(-18)+6`

Solution: `((5*Abs(-18))+6) = 96`

This page is intentionally left blank.
You may detach and use it as your exam script.